

The Real World of DEVOPS

How It Actually Works in Production
Enterprises | Startups | Tech Companies

THE COMPLETE OPERATIONAL GUIDE

Not Theory - Real Day-to-Day Operations
What Really Happens | Who Does What | How to Prepare

A Mental Model for Beginners

Written as if a Senior DevOps Architect Was Sitting Next to You

A Word Before You Begin

You probably came to this document after failing an interview question you did not understand. Or after months of reading about DevOps without feeling like you actually *get* it. That confusion is not because you are slow. It is because the way DevOps is usually taught is broken.

Tutorials teach you tools — install Jenkins, run Maven, write a Dockerfile. None of them show you the **actual production environment** where 200 developers commit code daily, where a deployment failure costs the company money every minute, where you wake up at 2am because your phone is ringing.

This document fixes that. It is written from the perspective of someone who has actually operated DevOps platforms in both regulated banks and tech startups. The goal is not to teach you tools — there are 10,000 tutorials for that. The goal is to give you the **mental model** of what real DevOps work looks like, so when you sit in your next interview or your first day on the job, the world makes sense.

HOW TO READ THIS: Read this document slowly. Each chapter builds on the last. By the end, you will know exactly what to learn, how to learn it, and what kind of job you are actually preparing for.

Table of Contents

PART 1 - THE TRUTH ABOUT DEVOPS

1. What DevOps Actually Is (And Is Not)	4
2. The Real Problem DevOps Solves	6
3. Tools vs Culture vs Process	7

PART 2 - THE REAL WORLD WORKFLOW

4. The 8 Phases of DevOps (The Infinity Loop)	9
5. A Real Feature from Idea to Production	11
6. The Pipeline in Detail (Every Stage)	14

PART 3 - TWO WORLDS

7. Enterprise DevOps - Banks, Insurance, Healthcare	17
8. Startup DevOps - Tech Companies, SaaS	20

PART 4 - WHO DOES WHAT

9. The DevOps Org Structure	23
10. L1, L2, L3 - What Each Level Actually Does	25
11. Specialized Roles - SRE, Platform, DevSecOps	27

PART 5 - A DAY IN THE LIFE

12. L1 Engineer - Hour by Hour	29
13. During an Incident - The 45-Minute Window	31

PART 6 - THE OPERATIONS LAYER

14. The Complete Toolchain (Visualized)	33
15. Monitoring, Logging, Alerting (Observability)	35

16. Change Management - How Code Reaches Production	37
---	----

PART 7 - YOUR PATH FORWARD

17. What to Learn (In Priority Order)	39
---------------------------------------	----

18. How to Build Real Experience	40
----------------------------------	----

19. Career Progression Roadmap	41
--------------------------------	----

PART 1

The Truth About DevOps

What DevOps Actually Is (And Is Not)

Strip away the jargon

DevOps is not a tool. It is not a job title. It is not Jenkins or Kubernetes or AWS. Those are tools that emerged because of DevOps. DevOps is a working model.

Before DevOps existed (2000s): developers wrote code, threw it over the wall to operations engineers, who then fought to deploy it. Developers blamed operations for slow deployments. Operations blamed developers for broken code. Releases happened monthly or quarterly. Production incidents took days to resolve. Everyone was unhappy.

What DevOps changed: developers and operations work together as one team, with shared responsibility for both building and running software. The same person who writes the code also cares about how it runs in production. Automation replaces manual handoffs. Software ships continuously instead of in big rare releases.

Three Things DevOps Combines

Component	What It Means	Reality Check
Culture	Developers + Operations work as one team. Shared ownership. No blame.	Hardest to change. Most companies fail at this part.
Practices	Continuous Integration, Continuous Delivery, Infrastructure as Code, monitoring.	These are the "how we work" rules.
Tools	Jenkins, Docker, Kubernetes, AWS, Terraform, Prometheus, etc.	The easy part. Tools are replaceable; the practices are not.

REAL WORLD INSIGHT: Companies that just buy tools without changing culture and practices fail at DevOps. You can have Jenkins, Kubernetes, and AWS and still be operating like it is 2005. When interviewers ask about DevOps, this is the depth they want to see.

Common Misconceptions

Misconception	Reality
"DevOps is a job title"	DevOps is a practice. Job titles vary: DevOps Engineer, SRE, Platform Engineer, Cloud Engineer.
"DevOps means no Ops team"	No. DevOps means the dev and ops teams collaborate. Ops still exists, just as part of the team.
"DevOps is just CI/CD"	CI/CD is one piece. DevOps also includes monitoring, infrastructure, incident management, security.
"If we have Jenkins, we have DevOps"	No. Jenkins is a tool. DevOps requires cultural change too.
"DevOps eliminates manual work"	It eliminates REPETITIVE manual work. Engineers still make judgment calls daily.

The Real Problem DevOps Solves

Why this exists at all

To understand DevOps, you need to understand the problem it solved. Picture a company before DevOps:

The Old World - A Story

A bank wants to add a new feature: customers can update their phone number through the mobile app. Today the process looks like this:

Week 1-2: Business analyst writes requirements. Architect designs solution.

Week 3-6: Developers write code. No automated tests. Manual integration with other teams.

Week 7: Code is "complete". Handed to QA team in a different building.

Week 8-10: QA finds 47 bugs. Sends back to dev. Cycle repeats.

Week 11: Code passes QA. Handed to Release Management team.

Week 12: Release Management schedules deployment for next monthly release window.

Week 14: Saturday night deployment. 6 engineers on call. Manual steps. 4 hour outage.

Week 15: Production bug found. Hotfix process takes another 2 weeks.

Total time from idea to live feature: 15+ weeks. Cost per release: tens of thousands of dollars in labor. Customer satisfaction: low. Developer satisfaction: low. Operations team: burned out.

What DevOps Did to This Process

Week 1: Product team writes ticket.

Week 2: Developer writes code WITH automated tests. Pushes to GitHub.

Day 1: CI pipeline runs automatically: build, test, security scan, deploy to dev environment.

Day 2: QA tests in dev (or automated tests catch everything). Approval.

Day 3: Same artifact promoted to staging. Smoke tests run.

Day 4: Same artifact deployed to production via blue-green. Zero downtime.

Day 5: Feature is live. Monitoring shows no issues.

Total time: 1-2 weeks. Cost: a fraction. Developers happy. Operations not burned out. Customers get features faster.

THE ANSWER THEY WANT: When the interviewer asks "Why is CI/CD important?" — this is the story. It is not about the tools. It is about cutting weeks to days, removing manual handoffs, catching bugs early, and shipping safely.

The Business Problems DevOps Solves

Business Problem	DevOps Solution
Slow time to market	Continuous deployment ships features in days not months
Production incidents take hours to fix	Automated rollback, monitoring catches issues fast
Deployments break things	Automated testing + staging mirrors prod + blue-green deployments

Compliance audits are painful	Pipeline as code = audit trail built in
Engineers leave (burnout)	Less firefighting, more building, predictable on-call
Security vulnerabilities reach production	SAST/SCA in pipeline blocks vulnerable code
Cannot scale to handle traffic	Cloud infrastructure + auto-scaling
Rolling back is risky	Versioned artifacts + immutable infrastructure = safe rollback

Tools vs Culture vs Process

The three legs of the DevOps stool

When you join a DevOps team, you will not just use tools. You will participate in practices and live a culture. All three matter equally.

The Three Legs

CULTURE (the foundation)

How people work together. The hardest to change, the most important.

- Developers and operations on same team
- Shared responsibility ("you build it, you run it")
- Blameless post-mortems (focus on systems, not people)
- Continuous learning (failures are learning opportunities)
- Customer focus (everything ties back to user value)
- Trust and autonomy (engineers make decisions)

PRACTICES (the methodology)

The "how we work" rules. Independent of tools.

- Continuous Integration (every push triggers build + tests)
- Continuous Delivery (every successful build is deployable)
- Infrastructure as Code (servers defined in Git)
- Monitoring everything (no blind spots in production)
- Trunk-based development or Gitflow branching
- Pair programming, code reviews, design reviews
- On-call rotations with proper handoffs

TOOLS (the enablers)

The technology that makes practices possible. Easily replaced.

- Source Control: GitHub, GitLab, Bitbucket
- CI/CD: Jenkins, GitHub Actions, GitLab CI, CircleCI
- Containerization: Docker, Podman
- Orchestration: Kubernetes, Docker Swarm
- Infrastructure as Code: Terraform, Ansible, CloudFormation
- Monitoring: Prometheus, Datadog, Splunk, New Relic
- Cloud: AWS, GCP, Azure

INTERVIEW TIP: When you are interviewing, talk about all three. Most candidates only talk about tools. Mentioning culture (blameless post-mortems, "you build it you run it") and practices (trunk-based development, monitoring) sets you apart.

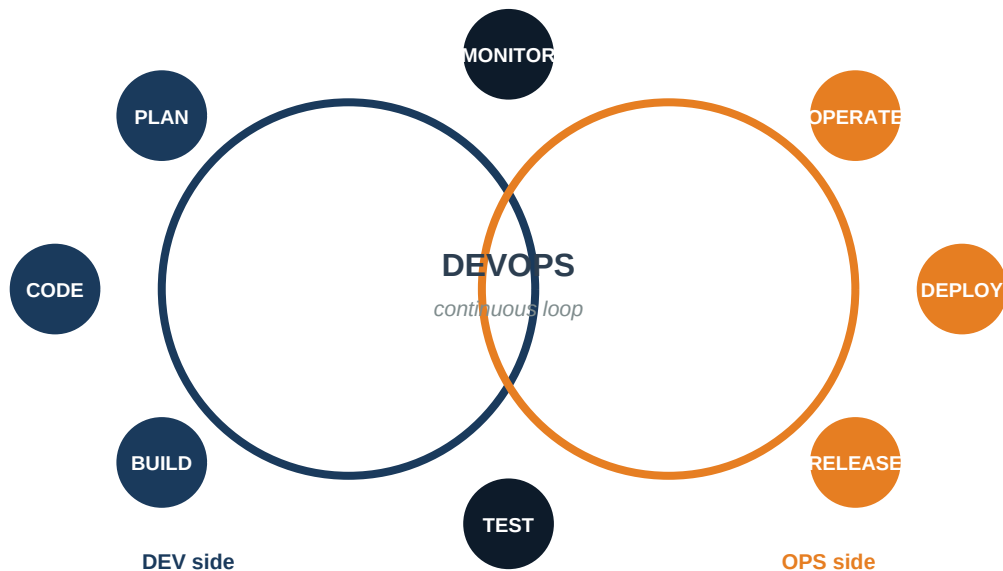
PART 2

The Real World Workflow

The 8 Phases of DevOps (The Infinity Loop)

The famous diagram, decoded

DevOps is often visualized as an infinity loop because the work never stops — code gets written, deployed, monitored, and new code is written based on what monitoring shows. It is a continuous cycle, not a linear process.



Each Phase in Detail

PLAN - Define what to build

Product managers, business analysts, and engineers decide what features to build next. Tickets are created in Jira/Linear/Asana. Sprint planning happens.

- Tools: Jira, Linear, Confluence, Miro
- DevOps role: minimal (mostly product/eng leadership)

CODE - Write the actual application

Developers write code on feature branches. They commit and push to GitHub.

- Tools: VSCode/IntelliJ, Git, GitHub
- DevOps role: provides Git workflows, branching strategy

BUILD - Compile and package code

CI system detects the push, runs the build (Maven, npm, Gradle), produces artifacts.

- Tools: Maven, Gradle, npm, Jenkins, GitHub Actions
- DevOps role: owns and maintains the build pipeline

TEST - Verify code works

Unit tests, integration tests, security scans (SAST), dependency scans (SCA).

- Tools: JUnit, pytest, SonarQube, OWASP Dependency Check
- DevOps role: configures and runs test infrastructure

RELEASE - Prepare for production

Artifact stored in Nexus/registry. Release notes generated. Change requests filed.

- Tools: Nexus, ECR, Docker Hub, ServiceNow
- DevOps role: manages artifact storage, release coordination

DEPLOY - Push to production

Code goes live via Kubernetes/EC2. Blue-green, canary, or rolling deployment strategy.

- Tools: Kubernetes, Helm, ArgoCD, Spinnaker, AWS CodeDeploy
- DevOps role: owns deployment pipelines and strategies

OPERATE - Run in production

Maintain the running system. Handle scaling, capacity, infrastructure changes.

- Tools: Terraform, Ansible, AWS Console, kubectl
- DevOps role: core operations work

MONITOR - Watch what is happening

Metrics, logs, traces. Alerts when things break. Dashboards show system health.

- Tools: Prometheus, Grafana, Datadog, Splunk, ELK
- DevOps role: builds and maintains observability stack

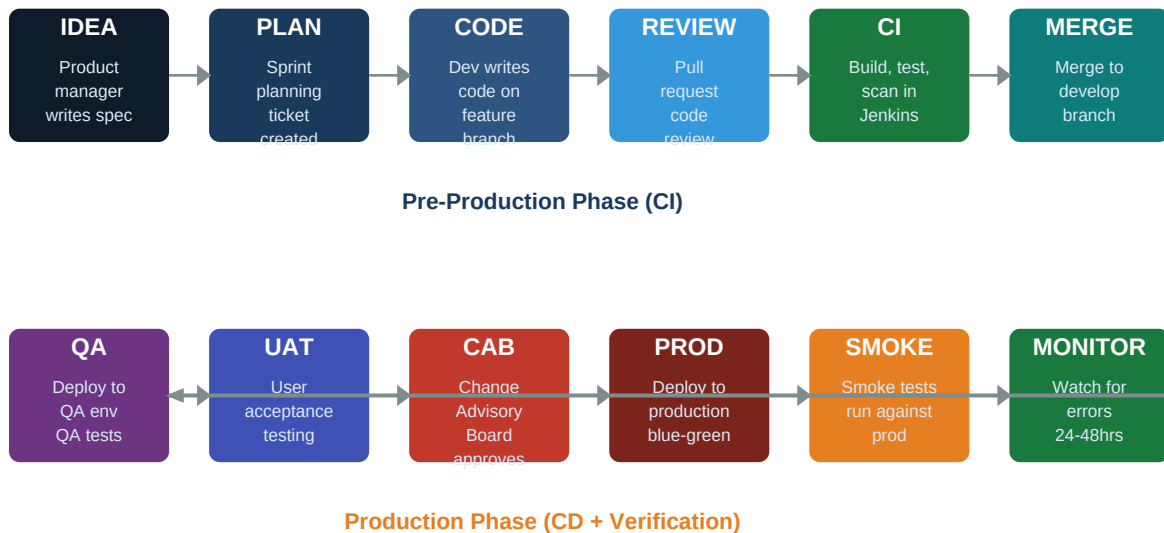
A Real Feature from Idea to Production

The complete journey of a piece of code

Let us trace one feature end to end. This is a real-world example from a large enterprise. Pay attention to who does what at each step.

Scenario: A bank wants to add a "Schedule Payment" feature so customers can schedule a payment for a future date through the mobile app.

A Real Feature Going from Idea to Production (Enterprise)



Total time enterprise: 2-6 weeks | Startup: 1-3 days

Step by Step Breakdown

Week 1: Idea & Planning

Product Manager writes user story: "As a customer, I want to schedule a payment for a future date so I do not forget bill payments." Jira ticket PAY-1234 created. Tech lead estimates 5 days. Added to next sprint.

Week 2 Day 1: Developer Starts Coding

Developer creates feature branch: feature/PAY-1234-schedule-payment from develop branch. Writes code locally. Tests locally. Pushes to GitHub.

Day 1-3: CI Pipeline Runs on Every Push

Each push to GitHub triggers Jenkins. Pipeline runs:

1. Code checkout
2. Maven build (mvn clean package)
3. Unit tests (JUnit)
4. SonarQube SAST scan + Quality Gate
5. OWASP Dependency Check (SCA)
6. Artifact published to Nexus (snapshot)
7. Docker image built and pushed to registry
8. Auto-deployed to DEV environment

If anything fails: developer is notified, fixes, pushes again. Cycle takes minutes.

Day 3: Pull Request

Developer raises PR: feature/PAY-1234 -> develop. Two senior engineers review code. Discussion in PR comments. Developer pushes fixes. PR pipeline runs on every push too. Eventually: 2 approvals + green pipeline = mergeable.

Day 3 (afternoon): Merge to Develop

PR merged. Develop branch CI pipeline runs. Artifact PAY-1234 is now in Nexus. Auto-deployed to SIT (System Integration Testing) environment.

Day 4: QA in SIT Environment

QA team runs manual test cases against SIT. They test happy path, edge cases, error handling. They find 1 bug. Reopen ticket. Dev fixes. New PR. New merge. New deployment to SIT. QA verifies. Closes ticket.

Day 5: UAT (User Acceptance Testing)

Business team and a few real customers test the feature in UAT environment. They approve the user experience. Sign-off in writing (banks require this).

Day 6: Change Advisory Board (CAB)

Weekly CAB meeting. Engineering lead presents PAY-1234. Discussion: any risk? Rollback plan? Monitoring in place? Communication plan? CAB approves. Deployment scheduled for Wednesday 2am production window.

Day 7: Production Deployment

Wednesday 2am. DevOps engineer triggers production pipeline. Same artifact (1.0.5) that has been tested in DEV, SIT, UAT is now promoted to PROD. Blue-green deployment: green environment receives new version. Smoke tests run. All pass. Load balancer switches traffic from blue to green. Deployment complete in 15 minutes.

Day 7-9: Production Monitoring

For the next 48 hours, the team watches dashboards. Are error rates higher than baseline? Is latency normal? Are customers using the feature? Any complaints in support tickets? Monitoring tools: Datadog, Splunk, support ticket dashboard.

Day 9: Stable Release

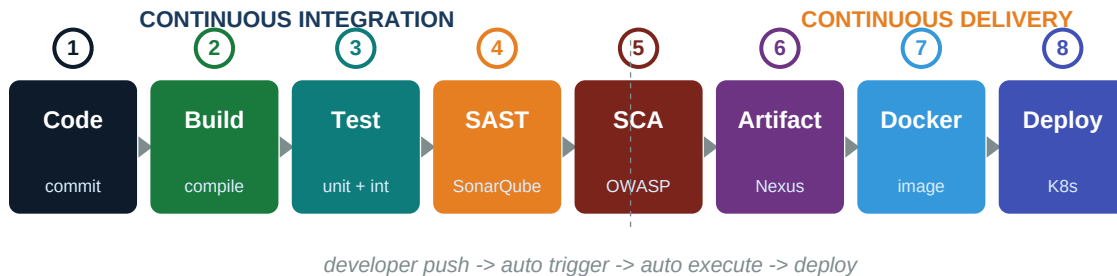
No issues found. PAY-1234 is officially considered a stable production release. Sprint review the next day: team celebrates the win.

ENTERPRISE vs STARTUP: In a startup, this same flow takes 1-3 days, not 9. Less ceremony, fewer approval gates, higher tolerance for fixing things after release. Different industries, different speeds.

The Pipeline in Detail

Every stage of CI/CD, explained

Now zoom into just the CI/CD pipeline. This is where DevOps engineers spend most of their building time. Every stage exists for a reason. Know each one.



Stage 1: CODE

Trigger: developer pushes code to GitHub branch.

Action: GitHub fires webhook -> Jenkins receives -> build queued.

Real concern: webhooks fail silently. DevOps verifies webhooks regularly.

Stage 2: BUILD

Action: Maven compiles .java -> .class, packages into .war/.jar.

Real failures: compilation errors, missing dependencies, OOM during build.

L1 action: notify developer with the exact error line from console.

Stage 3: TEST

Action: JUnit/TestNG runs unit tests + integration tests.

Real failures: flaky tests, environment issues, test data problems.

L1 action: distinguish "real failure" from "flaky test" - check test history.

Stage 4: SAST (SonarQube)

Action: scans source code for bugs, vulnerabilities, code smells. Quality Gate decides pass/fail.

Real failures: new vulnerabilities, coverage drop below threshold, code duplication.

L1 action: pull SonarQube report, notify developer with specific findings.

Stage 5: SCA (OWASP)

Action: scans dependency JARs against NVD CVE database.

Real failures: critical CVE in a library (Log4Shell scenario).

L1 action: create security ticket, notify security team, do not bypass.

Stage 6: ARTIFACT (Nexus)

Action: uploads .war to Nexus with version (e.g., payment-service-1.0.5.war).

Real failures: Nexus storage full, network issues, credentials expired.

L1 action: escalate to platform team, the issue is infrastructure not code.

Stage 7: DOCKER BUILD & PUSH

Action: docker build from Dockerfile, push to container registry.

Real failures: Docker daemon issues, registry full, image scan failures.

L1 action: check Docker daemon, registry quota, escalate if persistent.

Stage 8: DEPLOY TO K8S (Helm)

Action: helm upgrade deploys new image to Kubernetes cluster.

Real failures: pods CrashLoopBackOff, OOM, image pull errors, config issues.

L1 action: kubectl describe pod, kubectl logs, escalate per runbook.

PART 3

Two Worlds - Enterprise vs Startup

Large enterprises (think Citizens Bank, JPMorgan, Aetna, United Health, government agencies) operate DevOps differently than tech startups. Understanding this is critical because most DevOps jobs are at these companies.

The Constraints

Constraint	Why It Exists	How It Shapes DevOps
Regulatory compliance	SOX, PCI-DSS, HIPAA, FFIEC, GDPR	Every change is audited. Approval gates required.
Separation of duties	No single person can write code AND deploy it	Developers cannot deploy to production. Ops/DevOps team does.
Change management	Banks cannot have "move fast and break things"	Change Advisory Board, scheduled maintenance windows.
Long-lived systems	Mainframes, COBOL, 20-year-old apps	Modern pipelines must integrate with legacy systems.
Data privacy	Customer financial/health data is sensitive	Production access is heavily restricted. Test data is anonymized.
Risk aversion	A bad deploy can cost millions	Multiple environments (DEV, SIT, UAT, Pre-Prod, Prod). Slow promotion.

The Typical Stack

Layer	Common Tools in Enterprise
Source Control	GitHub Enterprise (self-hosted), Bitbucket, GitLab Enterprise
CI/CD	Jenkins (very common), GitLab CI, sometimes Bamboo
Build	Maven, Gradle (Java), .NET (Microsoft shops)
SAST	SonarQube, Fortify, Checkmarx, Veracode
SCA	Nexus IQ, Black Duck, Snyk Enterprise
Artifact Repo	Nexus, JFrog Artifactory
Containers	Docker (slow adoption), some still on VMs
Orchestration	Kubernetes (newer initiatives), VMs (legacy)
Cloud	AWS (most), Azure (Microsoft shops), often hybrid with on-prem
Monitoring	Splunk (logs), AppDynamics, Datadog, Dynatrace, New Relic

Ticketing	ServiceNow, Jira Service Management, Remedy
Change Mgmt	ServiceNow Change, custom internal tools

What L1 Actually Does at an Enterprise

Morning (8am): Check overnight build/deploy queues. Triage failed builds. Respond to support tickets.

Mid-morning: Handle developer requests (need access to X, need build env Y). Run pre-deployment checks.

Afternoon: Participate in change advisory meetings. Document runbooks. Help senior team with planned deployments.

End of day: Update incident tickets. Brief next shift. Update knowledge base.

On-call shifts: Respond to alerts, follow runbooks, escalate per SLA.

ENTERPRISE REALITY: Enterprise DevOps is highly process-driven. You will follow runbooks more than write them. You will escalate more than fix. You will document everything. This is correct behavior for L1 in regulated industries.

Tech startups (think Stripe, Notion, Linear, Vercel, Datadog itself, early-stage companies) operate DevOps very differently. Higher speed, less ceremony, more automation, more risk.

The Philosophy

Startup DevOps is driven by: **ship fast, fix faster**. The cost of not shipping is usually higher than the cost of a bad deployment. Engineers are trusted to deploy. Failures are expected and recovered from quickly.

What Is Different

ENTERPRISE <i>banks, insurance, healthcare</i>		STARTUP / TECH <i>product companies, SaaS</i>	
Deployment frequency	Weeks - Monthly	Deployment frequency	Multiple times daily
Pipeline stages	15-25 stages	Pipeline stages	5-10 stages
Approval gates	Multiple manual	Approval gates	Automated or none
Change advisory board	Required	Change advisory board	Not used
Compliance scans	Mandatory (SAST/SCA/DAST)	Compliance scans	Selective
Rollback time target	Under 1 hour	Rollback time target	Under 5 minutes
Production access	Restricted, audited	Production access	Broad but logged
Tools	Jenkins, Nexus, SonarQube	Tools	GitHub Actions, ECR, K8s
Team size	50-500+ engineers	Team size	5-100 engineers
Risk tolerance	Very Low	Risk tolerance	Higher (ship fast, fix fast)

The Typical Stack

Layer	Common Tools in Startups
Source Control	GitHub (cloud), GitLab, sometimes Bitbucket
CI/CD	GitHub Actions (very common), CircleCI, GitLab CI, ArgoCD
Build	Whatever the language uses: npm, pip, go build, cargo, etc.
SAST	Semgrep, Snyk Code, GitHub Advanced Security
SCA	Snyk, Dependabot, GitHub native scanning
Artifact Repo	Often skip - go straight to Docker registry
Containers	Docker (universal)
Orchestration	Kubernetes (EKS, GKE), Nomad, Fargate, sometimes just ECS

Cloud	AWS (dominant), GCP, sometimes multi-cloud
Monitoring	Datadog (very popular), Honeycomb, Grafana Cloud, New Relic
Logging	Datadog Logs, LogDNA/Mezmo, Papertrail, custom ELK
Incidents	PagerDuty, Opsgenie, Linear, Slack
IaC	Terraform (dominant), Pulumi, CDK
Secrets	AWS Secrets Manager, HashiCorp Vault, Doppler

What DevOps Looks Like at a Startup

Roles often blur: there may not be an L1 vs L2 distinction. Engineers do both.

Self-service: developers can deploy themselves. DevOps builds the platform.

Higher autonomy: a junior engineer might own a critical service end-to-end.

Faster cycles: deploy 5-50 times per day, not per month.

On-call rotation: all engineers (including devs) take turns being on-call.

Post-mortems: public, blameless, learning-focused.

Less compliance overhead: unless they handle payments (then PCI-DSS) or health data (HIPAA).

A Real Day at a SaaS Startup

9:00am: Standup over Slack. What did you ship yesterday? What today? Blockers?

9:30am: Pair with a backend dev on improving deployment time for their service.

11:00am: Review and merge a Terraform PR adding new RDS instance.

12:00pm: Lunch + Slack lurking.

1:00pm: Investigate elevated error rate alert from this morning. Find a regex change in a recent deploy. Roll it back via Argo. Notify the team. 30 minutes total.

2:00pm: Work on platform feature - making deployment pipeline 30% faster.

4:00pm: Help a new engineer debug their CI failure (env var typo).

5:00pm: Wrap up. On-call this week, so phone stays on.

STARTUP REALITY: Startup DevOps demands more autonomy, judgment, and breadth. You will be expected to figure things out independently. Less process, more thinking. Different skill profile than enterprise L1.

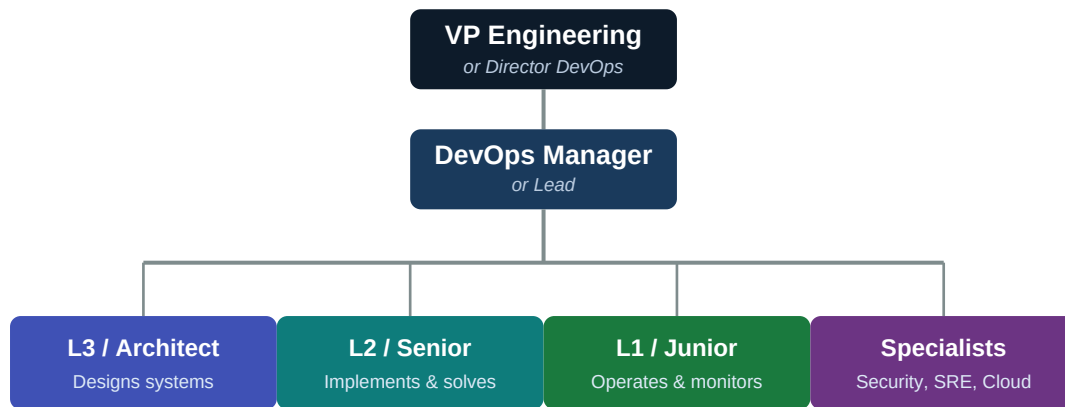
PART 4

Who Does What - The Roles

The DevOps Org Structure

How real teams are organized

Real DevOps teams have structure. Understanding the hierarchy helps you know where you fit, who reports to whom, and where to escalate.



*Real teams have 3-15 DevOps engineers across these levels
Larger orgs add SRE teams, Platform teams, Security (DevSecOps) teams*

Roles Explained

Role	Years Exp	Salary Range (US/India)	What They Do
L1 DevOps Engineer	0-2	\$60-90k / 5-10 LPA	Monitors, triages, runs runbooks, escalates
L2 DevOps Engineer	2-5	\$90-130k / 10-20 LPA	Builds pipelines, solves incidents, on-call leader
L3 / Senior DevOps	5-8	\$130-180k / 20-35 LPA	Designs platforms, mentors, owns major systems
Lead / Principal DevOps	8-12	\$180-250k / 35-50 LPA	Sets technical direction, hiring, cross-team
DevOps Architect	10+	\$200-300k+ / 40-80 LPA	Strategy, multi-team architecture, vendor decisions
Engineering Manager	8+	\$180-280k / 30-60 LPA	People management, team strategy, less hands-on
Director DevOps	12+	\$250-400k+ / 50-100 LPA	Multiple teams, organizational strategy

SALARY DATA: Salaries vary by location, company size, and industry. Banks and finance pay more than non-profits. SaaS startups offer equity. India salaries are listed in lakhs per annum (LPA). These are 2024 ranges - adjust for inflation.

L1, L2, L3 - What Each Level Actually Does

The clearest explanation you will read

L1 DevOps Engineer (0-2 years)

The first line of defense. Operational role.

Primary responsibility: Keep things running. Respond to alerts. Triage failures.

Daily activities:

- Monitor dashboards (Jenkins queue, Datadog, Splunk)
- Triage failed builds and notify developers
- Run pre-written runbooks for common issues
- Handle access requests (give developer X access to Y)
- Reset passwords, regenerate tokens
- Document incidents in tickets
- Escalate to L2 when stuck

Skills required: Basic Linux, basic CI/CD concepts, calm under pressure, good documentation.

Skills NOT required: Designing systems, writing complex pipelines, deep architecture.

Career path: L1 -> L2 typically 1-2 years.

L2 DevOps Engineer (2-5 years)

The doer. Hands-on technical role.

Primary responsibility: Build things. Solve complex incidents. Mentor L1.

Daily activities:

- Write and maintain Jenkins pipelines (Jenkinsfiles)
- Build Helm charts and Kubernetes manifests
- Write Terraform/Ansible for infrastructure
- Solve incidents that L1 escalated
- Lead on-call rotation, respond to P1 incidents
- Implement new monitoring and alerting
- Code reviews for L1 work
- Plan and execute production deployments

Skills required: Strong Linux, Docker, Kubernetes, Terraform, scripting (Bash, Python), one cloud provider deeply (AWS/GCP/Azure).

Career path: L2 -> L3 typically 2-3 years.

L3 / Senior DevOps Engineer (5-8 years)

The designer. Architecture and ownership role.

Primary responsibility: Design systems. Own major platforms. Technical leadership.

Daily activities:

- Architect CI/CD platforms (which tools, how they connect)
- Design Kubernetes cluster topology, networking, security
- Build internal developer platforms (self-service for devs)
- Choose tools, evaluate vendors, make strategic decisions
- Mentor L2 engineers and code review their work
- Write RFCs (Request for Comments) for major changes
- Lead post-mortems and drive systemic improvements
- Interview and hire L1/L2 engineers

Skills required: Everything L2 plus: system design, security, networking, organizational influence, written and verbal communication.

Career path: L3 -> Lead/Principal (technical track) or Manager (people track).

DevOps Architect (10+ years)

The strategist. Multi-team, multi-system role.

Primary responsibility: Define the entire DevOps strategy for the organization.

Daily activities:

- Set technology roadmap (where are we going in 2-3 years?)
- Standardize tooling across all teams
- Negotiate with vendors (millions of dollars at stake)
- Present to executives and the board
- Drive cross-team initiatives (e.g., migrate everyone to Kubernetes)
- Define SRE practices, SLOs, error budgets
- Influence engineering culture

Skills required: Deep technical breadth, business acumen, executive communication, strategic thinking, organizational politics.

Less hands-on coding. More writing, presenting, deciding.

Specialized Roles - SRE, Platform, DevSecOps

The DevOps family of jobs

As DevOps has matured, specialized roles have emerged. They all share DevOps principles but focus on different things. Understand them so you can target the right role.

Site Reliability Engineer (SRE)

Originated at Google. Focused on production reliability through engineering.

What they do:

- Define and measure Service Level Objectives (SLOs)
- Manage error budgets - how much downtime is acceptable
- Build automation to eliminate toil (repetitive manual work)
- Lead incident response and post-mortems
- Deep on-call rotations for critical services

How they differ from DevOps: SRE applies software engineering to operations. They write code to automate operations problems. DevOps engineers also do this, but SREs do it more deeply.

Found at: Google, Netflix, large tech companies.

Platform Engineer

Builds internal platforms that other engineers use.

What they do:

- Build self-service infrastructure (developers can spin up envs)
- Create internal CI/CD templates everyone uses
- Maintain Kubernetes clusters as a service
- Build internal developer portals (e.g., Backstage)
- Treat internal users as customers

How they differ from DevOps: Platform Engineering focuses on developer experience. DevOps engineers operate platforms; Platform engineers build them as products.

Found at: Mid-size to large companies, especially product-led companies.

DevSecOps Engineer

Security integrated into DevOps. Pipeline-embedded security.

What they do:

- Integrate security scans into CI/CD (SAST, SCA, DAST)
- Configure security tools (SonarQube, Snyk, NexusIQ)
- Manage secrets (Vault, AWS Secrets Manager)
- Container image scanning (Trivy, Twistlock)
- Compliance reporting for audits (SOX, PCI-DSS)
- Vulnerability management and CVE triage

How they differ from DevOps: Specialization in security. Often partners with security team, not just DevOps team. In regulated industries this is a senior role.

Found at: Banks, healthcare, government, large enterprises.

Cloud Engineer

Specialized in cloud infrastructure (AWS/GCP/Azure).

What they do:

- Design and build cloud architecture (VPCs, networking)
- Write Terraform/CloudFormation for infrastructure
- Optimize cloud costs (FinOps)
- Manage IAM, security groups, encryption
- Migrate from on-prem to cloud

How they differ from DevOps: Cloud-first focus. Less involved in CI/CD specifics. Deep expertise in one cloud platform.

Found at: Companies migrating to cloud, cloud-native startups.

Release Engineer / Build Engineer

Older term, still used in some enterprises.

What they do:

- Manage release processes and change calendars
- Coordinate cross-team deployments
- Maintain build infrastructure (Jenkins, Bamboo)
- Handle release notes and communication

How they differ from DevOps: Older role that DevOps largely subsumed. Still exists in conservative enterprises that have not fully modernized.

Found at: Traditional enterprises, banks, government.

PART 5

A Day in the Life

L1 Engineer - Hour by Hour

what you actually do all day

Here is a realistic day for an L1 DevOps engineer at a mid-size enterprise. Notice how much of the day is reactive (responding to things) versus proactive (building things). That is the nature of L1.

8:00 AM	Shift Handover Read handover notes from the night shift. Any open incidents? Any deployments still in progress? Any failed builds from overnight? Take notes.
8:15 AM	Morning Health Check Open monitoring dashboards: Jenkins build queue, agent health, Nexus storage, SonarQube server status. Check for any red indicators. All green? Good.
8:30 AM	Triage Failed Builds 12 builds failed overnight. Go through each: - 7 are developer errors (compilation, test failures) - notify each developer with link - 3 are flaky tests - re-run, mark as known issue - 2 are infrastructure (Nexus connection timeout) - escalate to L2
9:30 AM	Daily Standup 15-minute Zoom call with the DevOps team. Each person shares yesterday/today/blockers. You report on overnight triage. Note any work the team needs to take on.
9:45 AM	Access Request Queue ServiceNow has 8 access tickets: - 4 developers need Nexus read access (approve per policy) - 2 need Jenkins job-specific access (verify with manager) - 1 production read-only access (security team approval needed) - 1 unclear request (ask for clarification)
10:30 AM	Coffee + Slack Catch-up Break. Catch up on team Slack channels. Read engineering announcements.
11:00 AM	Routine Deployment to QA Team needs payment-service 1.0.7 deployed to QA. Standard runbook: 1. Verify artifact exists in Nexus 2. Trigger Jenkins deploy job with version=1.0.7 3. Watch deployment logs 4. Verify pod is healthy in QA cluster 5. Run smoke test endpoint 6. Notify team in Slack channel
12:00 PM	Lunch Eat. Stand up. Do not stare at logs while eating.

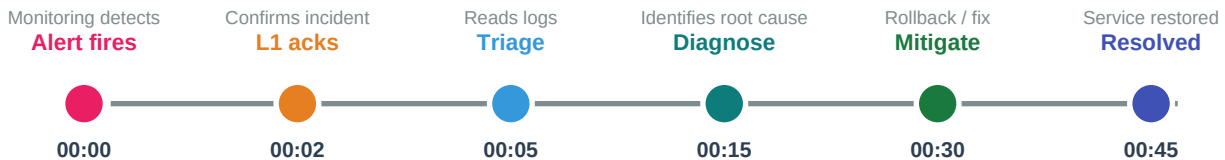
1:00 PM	Documentation Update Manager assigned a Confluence page update - the Nexus runbook is outdated. Update it based on the current Nexus version. Have a peer review it.
2:00 PM	SonarQube Quality Gate Failures 5 quality gates failed today. Pull each report. Send specific findings to the developers. For 1 finding (Critical SQL injection), also notify the security team.
3:00 PM	INCIDENT - Nexus Slow Alert: Nexus response times above threshold. Multiple builds timing out. - Acknowledge the alert in PagerDuty - Post in incident channel: "Investigating Nexus slowness" - Check Nexus dashboards: disk at 85% full - Escalate to L2: "Nexus disk at 85%, need urgent storage cleanup or expansion" - L2 takes over. You provide context and stand by. - Resolution: L2 ran cleanup policy, deleted old SNAPSHOT versions. Disk at 65%. - Update incident ticket with timeline and resolution. Close.
4:30 PM	Knowledge Base Search Improvement Free time! Work on a small project to improve the team's runbook search by adding better tags. Submit PR to docs repo.
5:30 PM	Shift Handover Notes Write handover for next shift: - 12 overnight builds triaged, 5 dev notifications sent - Nexus disk incident resolved, L2 implemented cleanup - 5 SonarQube QGs notified, 1 escalated to security - All systems green at handoff
6:00 PM	End of Shift Log off. The next shift continues from your notes.

During an Incident - The 45-Minute Window

How incidents actually unfold

When production breaks, every minute counts. Companies measure their incident response in MTTR (Mean Time To Recovery). Target: under 45 minutes for most incidents.

Incident Response Timeline (target: under 45 min)



After: post-mortem within 24 hours, action items tracked

Real Incident Walkthrough

Scenario: It is 2pm. The payment-service API starts returning 500 errors. Customer support tickets spike. This is a P1 incident.

14:00 - Alert Fires

Datadog detects error rate above 5% on payment-service. Alert fires to PagerDuty. On-call engineer phone rings.

14:02 - L1 Acknowledges

L1 engineer on-call acknowledges in PagerDuty within 5 minutes (SLA requirement). Opens laptop, joins incident Slack channel: #inc-payment-api-down.
Posts: "ACK - investigating payment-service errors. Initial assessment in 5 min."

14:05 - Triage Begins

L1 checks:

- Datadog dashboard: error rate at 23% and rising
- Recent deployments: payment-service 1.0.8 deployed 14:00 - matches timing
- Pod health: all pods Running but error logs show "DB connection refused"

L1 posts in channel: "payment-service 1.0.8 deployed at 14:00, errors started at 14:01. Logs show DB connection issues. Suspect new release."

14:10 - Decision: Rollback

L1 follows runbook: "If new deployment correlates with incident, rollback immediately."
Posts in channel: "Initiating rollback to 1.0.7 per runbook."
Notifies L2 on-call: "Rolling back payment-service. Will update in 5 min."

14:12 - Rollback Initiated

L1 triggers Jenkins rollback job:
Jenkins -> payment-service-rollback -> version=1.0.7 -> Build
Or via Helm:
helm rollback payment-service 12 -n production
Watches deployment logs.

14:15 - L2 Joins

L2 on-call joins the channel. Reviews timeline. Confirms L1's decision was correct. L2 takes over incident command. L1 continues to execute the rollback.

14:18 - Rollback Complete

Pods running 1.0.7. Error rate dropping rapidly.
L1 posts: "Rollback complete. Error rate dropping. Verifying."

14:22 - Service Recovered

Error rate back to baseline 0.1%. Datadog alert resolves.
L1 posts: "RESOLVED - service recovered. Error rate normal. Will continue to monitor for 30 min."

14:25 - Communication

L2 notifies stakeholders:
- Engineering manager: incident summary
- Customer support: deploy info to share with customers if asked
- Status page: incident updated to RESOLVED

14:30-15:00 - Monitor

L1 keeps eyes on dashboards for 30 min to ensure stability. No further issues.

Next day - Post-Mortem

Within 24-48 hours, the team holds a blameless post-mortem:

- What happened? (timeline)
- What was the root cause? (DB connection pool sizing wrong in 1.0.8)
- Why did it pass testing? (load test environment had different DB)
- Action items:
 - Add connection pool config to standard test suite
 - Add canary deployment (5% traffic first) to catch this earlier
 - Update runbook with this incident type
 - Document published. Actions tracked in Jira.

THE L1 LESSON: Notice: L1 did not need to understand WHY the new code broke. They followed the runbook: "recent deploy + matching timing = rollback." That is the operational discipline you are being hired for. L2 and the developer figure out the root cause later.

PART 6

The Operations Layer

The Complete Toolchain (Visualized)

Every tool, organized by purpose

DevOps has dozens of tools. New ones appear every year. Instead of trying to learn every tool, learn the CATEGORIES of tools and know one strong example of each. You can pick up a new tool in the same category quickly.

Source Control	Where code lives
<i>Git is universal. The hosting platform varies.</i>	
GitHub (cloud), GitLab (cloud/self-hosted), Bitbucket (Atlassian), Gitea (self-hosted), GitHub Enterprise (banks)	
CI/CD Orchestration	Runs pipelines
<i>The brain of automation.</i>	
Jenkins (enterprises), GitHub Actions (startups), GitLab CI (mid-market), CircleCI, ArgoCD (Kubernetes-native), Spinnaker (Netflix), Tekton, Bamboo (legacy)	
Build Tools	Compile and package code
<i>Language-specific.</i>	
Java: Maven, Gradle Python: pip, poetry Node: npm, yarn Go: go build Rust: cargo .NET: dotnet C/C++: make, cmake	
Testing	Verify code works
<i>Unit tests by language.</i>	
Java: JUnit, TestNG Python: pytest, unittest Node: Jest, Mocha Integration: Selenium, Cypress, Postman Performance: JMeter, k6, Locust	
Code Quality / SAST	Find bugs in source code
<i>Static analysis.</i>	
SonarQube (most common in enterprise), Semgrep (modern, popular in startups), Fortify (Micro Focus), Checkmarx, Veracode, GitHub CodeQL	
Dependency Scanning / SCA	Find CVEs in libraries
<i>Critical for security.</i>	
OWASP Dependency Check (free, open source), Snyk (commercial), Nexus IQ (Sonatype), Black Duck (Synopsys), Dependabot (GitHub free), Renovate	
Artifact Repository	Store built artifacts
<i>Versioned storage.</i>	
Java: Nexus, JFrog Artifactory Container images: Docker Hub, AWS ECR, Google GCR, Azure ACR, Harbor (self-hosted) Generic: Nexus, Artifactory	
Containerization	Package apps with environment

Docker dominates.

Docker (universal), Podman (RedHat alternative), Buildah, Buildpacks (Heroku-style)

Container Orchestration

Run containers at scale

Kubernetes is the standard.

Kubernetes (the standard), AWS ECS (simpler), Docker Swarm (legacy), Nomad (HashiCorp), managed K8s: AWS EKS, GCP GKE, Azure AKS

Infrastructure as Code

Define infrastructure in code

Replaces clicking in cloud consoles.

Terraform (most popular, multi-cloud), AWS CloudFormation (AWS only), Ansible (config mgmt + IaC), Pulumi (uses Python/TypeScript), Chef, Puppet (older)

Configuration Management

Manage server configs

Less common since containers.

Ansible (most popular), Chef, Puppet, SaltStack

Monitoring & Metrics

Watch what is happening

Time-series data + dashboards.

Prometheus + Grafana (open source standard), Datadog (paid, popular), New Relic, AppDynamics (enterprise APM), Dynatrace, CloudWatch (AWS), Azure Monitor

Logging

Aggregate logs from everywhere

Search across thousands of servers.

ELK Stack (Elasticsearch + Logstash + Kibana - self-hosted), Splunk (enterprise, expensive), Datadog Logs, Loki (Grafana), CloudWatch Logs (AWS)

Distributed Tracing

Follow requests across services

Microservices debugging.

Jaeger (open source), Zipkin (older), Datadog APM, New Relic, OpenTelemetry (standard)

Alerting

Wake people up when things break

On-call management.

PagerDuty (most popular), Opsgenie (Atlassian), VictorOps, native cloud (CloudWatch Alarms)

Secrets Management

Store passwords, tokens, keys

Never in code.

HashiCorp Vault (enterprise gold standard), AWS Secrets Manager, Azure Key Vault, GCP Secret Manager, Doppler, 1Password Secrets

Service Mesh

Manage service-to-service communication

Advanced K8s pattern.

Istio (most features), Linkerd (simpler), Consul Connect, AWS App Mesh

Cloud Platforms

Where everything runs

Three giants dominate.

AWS (40%+ market share), Microsoft Azure (banks, .NET shops), Google Cloud (newer, data-focused), Oracle Cloud, IBM Cloud (legacy enterprises)

Monitoring, Logging, Alerting (Observability)

How you know what is happening in production

You cannot operate what you cannot see. Observability is the practice of making systems visible. It has three pillars: metrics, logs, traces.

The Three Pillars of Observability

Pillar	What it tells you	Example tools
Metrics	Numbers over time: CPU, memory, request rate, error rate	Prometheus, Datadog, CloudWatch
Logs	Text events: what your application is doing, errors	ELK, Splunk, Datadog Logs
Traces	Follow a single request across multiple services	Jaeger, Datadog APM, OpenTelemetry

The 4 Golden Signals (Google SRE)

Google SREs identified 4 metrics that tell you almost everything about service health:

Signal	What It Measures	Why It Matters
Latency	How long requests take	Slow service = bad user experience
Traffic	How many requests per second	Drop = users cannot reach you. Spike = potential overload.
Errors	Rate of failed requests	Direct measure of service health
Saturation	How "full" your service is (CPU, memory, queue)	Approaching limits = imminent failure

What Real Monitoring Looks Like

Service-level dashboards: one dashboard per service showing the 4 golden signals.

Infrastructure dashboards: servers, databases, message queues, network.

Business dashboards: signups per hour, orders processed, revenue per minute.

SLO dashboards: are we meeting our reliability targets?

Alerting - The Hard Part

Bad alerts are worse than no alerts. Bad alerts: 50 emails per day, all ignored. Good alerts: 1-2 per week, all actionable.

Good alert characteristics:

- Symptom-based (user-facing impact), not cause-based (CPU high)
- Actionable (clear runbook to follow)
- Time-sensitive (needs human in the next 30 min)
- Rare (does not fire daily)

Bad alert characteristics:

- "CPU above 70%" (so what? users still happy)
- "Disk at 85%" (might be fine for hours)
- "Memory increased" (normal during traffic)

These should be dashboard metrics, not alerts.

REAL OPERATIONAL WISDOM: When you join a team, ask: "How many alerts do we get per week? How many are real?" If it is more than 5 per engineer per week, the team has alert fatigue and is missing real incidents because everyone ignores them. Fixing this is high-value DevOps work.

Change Management - How Code Reaches Production

The path from develop branch to live

In enterprises, code goes through multiple environments before reaching production. Each environment serves a purpose. Same code (same artifact), promoted environment to environment.

The Standard Environment Hierarchy

Environment	Purpose	Who Uses It	Data
DEV	Developers test their work	Developers	Synthetic
SIT (System Integration)	Multiple services tested together	Developers + QA	Synthetic
QA / Test	Formal testing by QA team	QA engineers	Anonymized prod data
UAT (User Acceptance)	Business users approve	Business users	Anonymized prod data
Pre-Prod / Staging	Production-like for final tests	Limited (release team)	Anonymized or live mirror
PROD	Real customers	Customers	Real data

IMMUTABLE ARTIFACT PRINCIPLE: Critical rule: the SAME artifact (.war/.jar/docker image) that was tested in DEV should be the one deployed to PROD. Never rebuild for production. Always promote. This eliminates "works in DEV, breaks in PROD" issues.

Deployment Strategies

Strategy	How It Works	When to Use
Big Bang	Stop old, start new. Downtime.	Only when downtime is acceptable. Avoid.
Rolling Update	Replace pods one at a time. K8s default.	Most apps. Default in Kubernetes.
Blue-Green	Two identical environments. Switch traffic.	When zero downtime is required. Easy rollback.
Canary	5% traffic to new version, monitor, then 25%, 100%	Risky changes. Catch issues before full rollout.
A/B Testing	Different versions to different user segments	Testing features. Not for infrastructure.
Shadow	Send copy of real traffic to new version (no response sent)	Validate performance before exposing to users.

Change Advisory Board (CAB)

In regulated industries, major changes must be approved by a Change Advisory Board. This is a weekly or daily meeting where:

- Each change is presented with: risk assessment, rollback plan, testing evidence
- Stakeholders ask questions and approve or reject
- Approved changes are scheduled for specific maintenance windows
- All decisions are recorded for audit

CABs slow things down but exist for a reason - banks cannot have surprises. As DevOps engineer, you may present at CAB meetings. Practice clear, concise risk explanations.

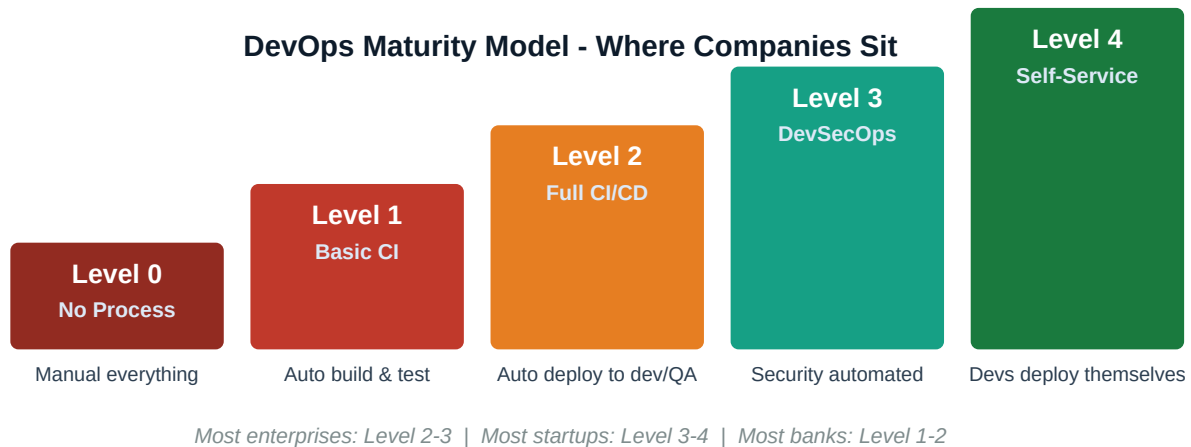
PART 7

Your Path Forward

What to Learn (In Priority Order)

Stop learning randomly

The DevOps landscape is huge. You cannot learn everything. Here is the priority order that maximizes your job-readiness fastest.



Priority Order for L1 DevSecOps Job

PRIORITY 1: Foundation (must have)

- Linux command line - really comfortable, not just knowing commands
- Git - daily workflow, branching, merging, rebasing, resolving conflicts
- Basic networking - HTTP, DNS, TCP, ports, IPs, subnets
- One scripting language - Bash or Python at a basic level
- Cloud basics - AWS EC2, S3, IAM, security groups

Time investment: 2-3 months if no prior knowledge

PRIORITY 2: Core DevOps (gets you the job)

- CI/CD concepts - what each stage does and why
- Jenkins - install, configure, write Jenkinsfile, debug pipelines
- Maven (for Java work) - pom.xml, build lifecycle, dependencies
- Docker - images, containers, Dockerfile, multi-stage builds
- Basic Kubernetes - pods, deployments, services, kubectl
- Monitoring basics - metrics, logs, alerts (read dashboards)

Time investment: 3-4 months

PRIORITY 3: DevSecOps Specific (your target role)

- SonarQube - SAST concepts, Quality Gates, reading reports
- OWASP Dependency Check - SCA concepts, CVE triage
- Security gate enforcement - why and how
- Vulnerability management workflow
- Secrets management - Vault basics, AWS Secrets Manager

Time investment: 2 months

PRIORITY 4: Make You Senior-Ready

- Terraform - infrastructure as code
- Helm - Kubernetes package manager
- Advanced Kubernetes - networking, RBAC, troubleshooting
- Prometheus + Grafana - building monitoring
- One CI/CD alternative (GitHub Actions or GitLab CI)

Time investment: 6+ months (after job, in the role)

PRIORITY 5: Bonus / Advanced

- ArgoCD or Spinnaker (GitOps)
- Service Mesh (Istio)
- Multi-cloud (Azure or GCP in addition to AWS)
- AWS certifications (Solutions Architect Associate)
- Programming - become fluent in Python or Go

Time investment: ongoing throughout career

How to Build Real Experience

Without a job yet

You cannot get a real DevOps job without experience. But you cannot get experience without a job. The way out: build experience yourself by simulating real environments.

Build These 5 Things (In Order)

Project 1: Complete Java CI/CD Pipeline

What you build:

- A Spring Boot Hello World app in GitHub
- Jenkins on AWS EC2 with full pipeline (build, test, SonarQube, Nexus, Docker, deploy)
- Document everything in a README

What it teaches: end-to-end pipeline ownership.

Time: 2-3 weeks.

Talking point: "I built a complete CI/CD pipeline including security scans..."

Project 2: Kubernetes Deployment

What you build:

- Deploy the Spring Boot app to Kubernetes (Minikube or EKS)
- Write Helm chart with values for dev/staging/prod
- Add a database (PostgreSQL) as a separate deployment
- Configure Ingress for HTTPS

What it teaches: container orchestration basics.

Time: 2-3 weeks.

Talking point: "I deployed a Spring Boot app to Kubernetes with Helm..."

Project 3: Infrastructure as Code

What you build:

- Terraform to provision EC2, VPC, security groups, IAM roles
- Modular Terraform (modules for VPC, EC2, RDS)
- State file in S3 with DynamoDB locking

What it teaches: cloud infrastructure as code.

Time: 2 weeks.

Talking point: "I built our infrastructure as code using Terraform..."

Project 4: Monitoring Stack

What you build:

- Prometheus + Grafana on Kubernetes
- Instrument the Spring Boot app to expose metrics
- Build 4 golden signal dashboard
- Configure 3 alerts (high error rate, high latency, low success)

What it teaches: real observability skills.

Time: 2 weeks.

Talking point: "I implemented Prometheus monitoring with custom dashboards..."

Project 5: Break Things Intentionally

What you do:

- Intentionally fill up Nexus disk - watch what fails, fix it
- Kill Jenkins agent mid-build - watch what happens to job
- Push a vulnerable dependency, see OWASP catch it, fix it
- Deploy bad code, practice rollback procedure
- Simulate Nexus down, see how pipeline reacts

What it teaches: operational thinking. THIS IS THE MISSING PIECE.

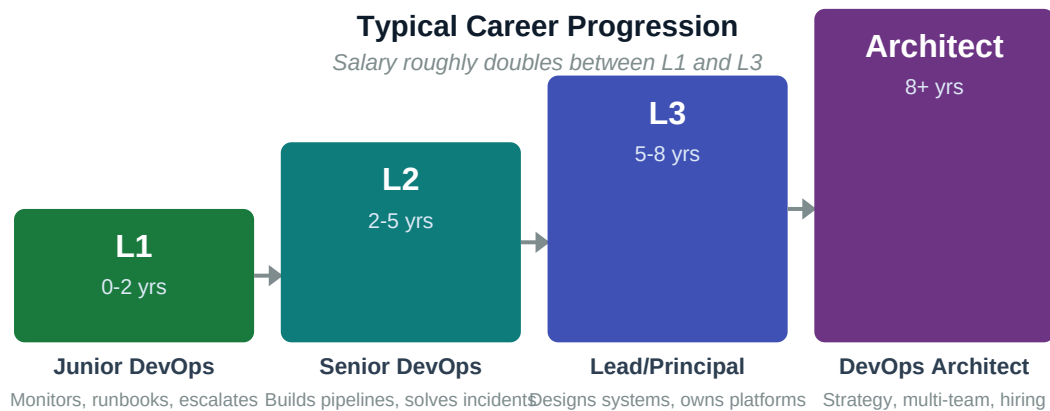
Time: ongoing.

Talking point: "I deliberately broke my CI/CD setup to learn how to fix common issues..."

THE SECRET WEAPON: Project 5 is the most important and the one nobody does. Most candidates have built a pipeline. Almost none have intentionally broken one and fixed it. That gives you the operational stories that win interviews.

Career Progression Roadmap

Where this leads



How to Move Up

From L1 to L2 (1-2 years)

What to demonstrate:

- Can handle most incidents without escalation
- Can write basic pipeline modifications
- Can mentor newer L1 engineers
- Have built side projects beyond your job

Skills to add:

- Write your own Jenkinsfiles (not just maintain)
- Terraform basics
- One scripting language fluently (Python or Bash)
- Deep dive on one tool (e.g., become the team Kubernetes expert)

From L2 to L3 (2-3 years)

What to demonstrate:

- Owned a major project end to end
- Led incident response as commander
- Mentored multiple junior engineers
- Written technical documents read by other teams

Skills to add:

- System design thinking
- Cost optimization (FinOps basics)
- Security at architecture level
- Cross-team communication skills

From L3 to Architect (3-5 years)

What to demonstrate:

- Led major migrations (e.g., on-prem to cloud)
- Made decisions that affected multiple teams
- Influenced engineering culture
- Written and given conference talks or blog posts

Skills to add:

- Strategic thinking (what should we do in 2 years?)
- Vendor evaluation and negotiation
- Executive communication
- Industry knowledge (where is DevOps going?)

Final Words

DevOps is a great career. Strong demand, good pay, intellectually engaging, and you are always building something. But it requires real, hands-on operational knowledge - not just theory.

Your last interview failure was a gift. It told you exactly what you did not know. Use that. Build the projects in Chapter 18. Especially Project 5 (intentionally break things). Within 2-3 months you will have stories that pass any L1 interview.

You are not behind. You are exactly where you should be when you start asking the right questions. The right questions started with this document. The next step is doing the work.

"The path is made by walking it."

This document is reference material. Read it multiple times. The concepts will become clearer with each reading. Pair it with hands-on projects, and you will have the mental model and the skills to succeed.